

Plano de Testes - API ServeRest

Autor: Gustavo Santos Queiroz

Ferramentas: Postman, Confluence, Xmind.

1. Escopo e Objetivos

- **Objetivo:** Validar a funcionalidade, segurança básica e integridade dos dados da API ServeRest, garantindo que os fluxos de cadastro de usuários, gestão de produtos e carrinhos de compra funcionem conforme a documentação.
- **Escopo:**
 - CRUD de Usuários (/usuarios).
 - Autenticação (/login).
 - Gestão de Produtos (/produtos).
 - Fluxo de Carrinho de Compras (/carrinhos).
- **Fora do Escopo:**
 - Testes de carga/performance.
 - Front-end da aplicação Serverest.

2. Mapeamento dos Endpoints

Recurso	Método	Endpoint	Descrição e Regras
Login	POST	/login	Autentica o usuário (Email/Senha). Retorna um Bearer Token com validade de 10 minutos.
Usuários	GET	/usuarios	Lista todos os usuários. Aceita filtros (nome, email, etc).
	POST	/usuarios	Cadastra novo usuário. Email deve ser único. Pode ser Admin (true) ou Comum (false).
	GET	/usuarios/{_id}	Busca um usuário específico pelo ID.
	PUT	/usuarios/{_id}	Edita um usuário. Se o ID não existir, cria um novo usuário. Email deve ser único.
	DELETE	/usuarios/{_id}	Remove um usuário. Bloqueia a exclusão se o usuário tiver carrinho cadastrado.
Produtos	GET	/produtos	Lista todos os produtos. Aceita filtros (nome, preço, etc).
	POST	/produtos	Cadastra novo produto. Exige Token de Admin. Nome deve ser único.

	GET	/produtos/{_id}	Busca um produto específico pelo ID.
	PUT	/produtos/{_id}	Edita um produto. Exige Token de Admin.
	DELETE	/produtos/{_id}	Remove um produto. Exige Token de Admin. Bloqueia se o produto estiver em carrinho.
Carrinhos	GET	/carrinhos	Lista todos os carrinhos ativos no sistema.
	POST	/carrinhos	Cria carrinho e vincula produtos ao usuário. Exige Token. Limite de 1 carrinho por usuário. Abate do estoque.
	GET	/carrinhos/{_id}	Busca um carrinho específico pelo ID.
	DELETE	/carrinhos/concluir-compra	Finaliza a compra e exclui o carrinho. Exige Token. Ocorre apenas se o Token for do dono do carrinho.
	DELETE	/carrinhos/cancelar-compra	Cancela a compra e exclui o carrinho. Exige Token. Devolve os produtos para o estoque.

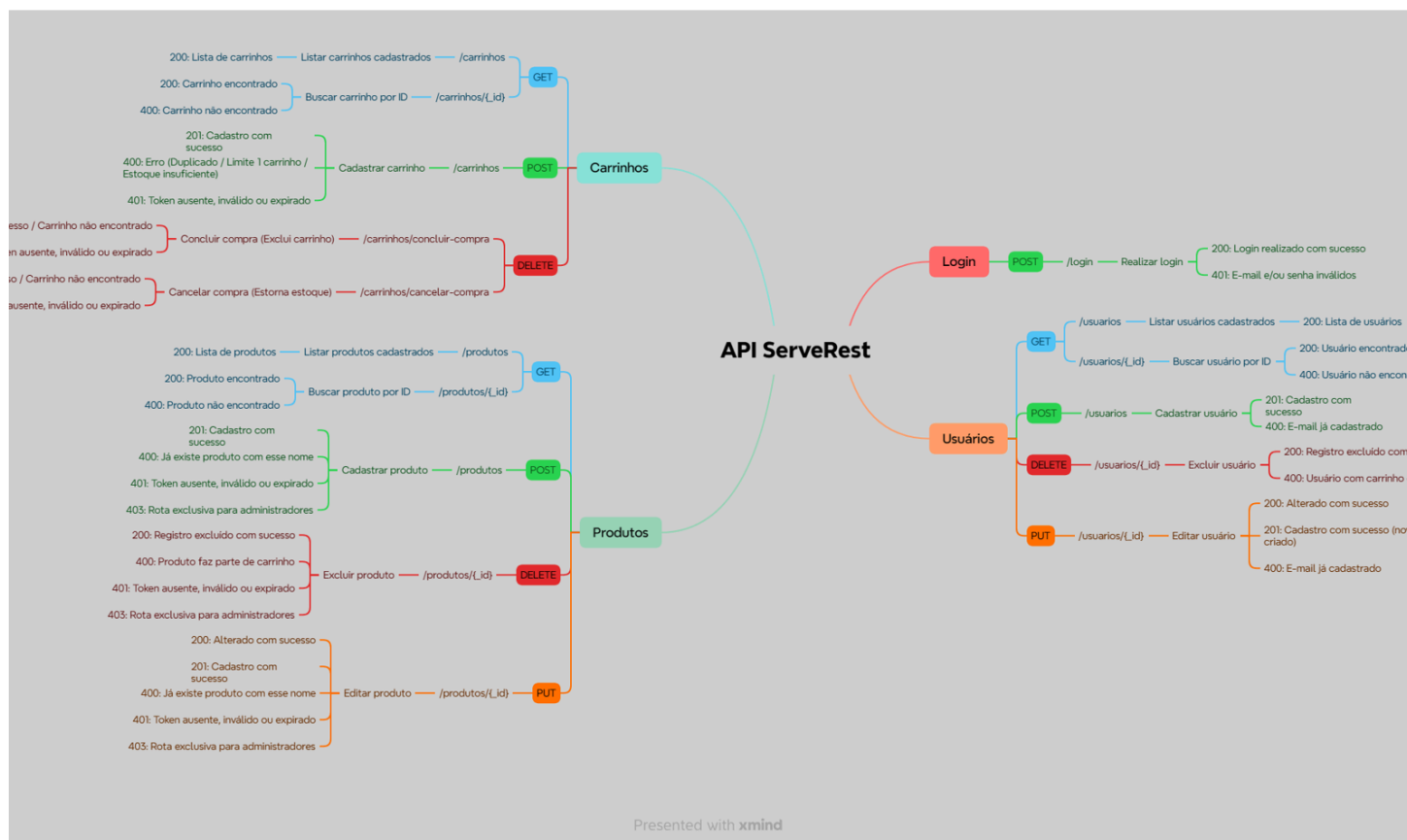
3. Riscos e Prioridades

Risco Identificado	Impacto	Prioridade	Mitigação Resumida
Falhas de Autenticação e Segurança (Token inválido/expirado, Acesso indevido de não-admin)	Crítico	Alta	Automatizar testes de validação de Token, checar expiração (10min) e tentar acessar rotas de Admin com usuário comum.
Inconsistência no Fluxo de Compra e Estoque (Erro no cálculo de baixa/estorno, Múltiplos carrinhos)	Alto	Alta	Validar a reposição do estoque (/cancelar-compra) e a regra de carrinho único por usuário.
Violação de Integridade e Validação de Dados	Médio	Média	Executar testes negativos tentando duplicar cadastros

(Duplicidade de e-mail/nome,
Exclusão de registros em uso)

únicos e excluir
usuários/produtos vinculados a
carrinhos ativos.

4. Mapa Mental



5. Estratégia de Testes Exploratórios

- **Técnica Escolhida:** Heurística de "Tour do Supermercado".
- **Justificativa:** A API simula um e-commerce. A abordagem do "Tour" permite simular o fluxo real de um usuário navegando, escolhendo produtos e fechando a compra, em vez de testar endpoints isolados, garantindo a integração entre os recursos.

6. Cenários de Teste

ID	Título do Cenário	Tipo	Endpoint	Dados de Entrada	Resultado Esperado	Status Code
CT01	Cadastrar Usuário Administrador	Funcional (Positivo)	POST <code>/usuarios</code>	Nome, Email único, Senha, Administrador="true" .	Usuário criado com sucesso e ID retornado. (Necessário ser Admin para criar produtos no CT03).	201 Created
CT02	Realizar Login (Obter Token)	Funcional (Autenticação)	POST <code>/login</code>	Email e Senha cadastrados no CT01.	Login realizado com sucesso. O corpo da resposta deve conter o Token Bearer obrigatório para	200 OK

					os próximos passos.	
CT03	Cadastrar Produto no Estoque	Funcional (Pré-condição)	POST /produtos	Header: Token (CT02). Body: Nome único, Preço, Descrição, Qtd.	Produto criado com sucesso. Retorna o ID do produto (usado no carrinho).	201 Created
CT04	Listar Produto via ID	Funcional (Validação)	GET /produtos	ID do produto.	A lista deve retornar o produto criado no CT03, confirmando que ele está visível na loja.	200 OK
CT05	Adicionar Item ao Carrinho	Funcional (Negócio)	POST /carrinhos	Header: Token (CT02). Body: ID do Produto (CT03) e Quantidade (1).	O item deve ser vinculado ao usuário e a quantidade deve ser reservada do estoque.	201 Created
CT06	Concluir Compra (Checkout)	Funcional (Fluxo)	DELETE /carrinhos/ concluir- compra	Header: Token (CT02).	A compra deve ser finalizada, o carrinho esvaziado e o estoque (já baixado no CT05) confirmado.	200 OK

7. Critérios de Entrada, Saída e DoD

• Critérios de Entrada:

- Pré-requisitos para que a fase de execução dos testes possa começar:
 - **Ambiente:** API ServeRest online, acessível e estável.
 - **Documentação:** Swagger acessível para consulta de contratos.
 - **Ferramentas:** Postman instalado e collection base importada.
 - **Massa de Dados:** Estratégia de dados definida (ex: uso de variável do Postman para gerar e-mails únicos ou scripts de Pre-request para criação de massa).

• Critérios de Saída:

- Condições para encerrar o ciclo de testes:
 - Cenários planejados foram executados.
 - Cenários de prioridade Crítica ou Alta testados.
 - Nenhum bug de severidade Crítica ou Bloqueante em aberto. Bugs de severidade "Baixa" devidamente documentados.
 - Todos os bugs corrigidos durante o ciclo foram retestados e fechados.

• Definição de Pronto (DoD):

- O que define que o ciclo foi concluído com qualidade:
 - ☑ Plano de testes revisado e aprovado.
 - ☑ Collection do Postman estruturada, com variáveis de ambiente configuradas e scripts implementados.
 - ☑ Automação dos testes funcionais principais (Happy Path) concluída.
 - ☑ Relatório de execução gerado (com evidências de sucesso e falha).
 - ☑ Bugs reportados na ferramenta de gestão (Jira) com passos para reprodução e evidências.

8. Evidências e Modelos de Requisição

CT01: Cadastrar Usuário Administrador

- **Método:** POST
- **URL:** `{{baseUrl}}/usuarios`
- **Body (JSON):**

```
1 {
2   "nome": "Gustavo Teste",
3   "email": "{{email_atual}}",
4   "password": "teste",
5   "administrador": "true"
6 }
```

- **Resposta:**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "KSgSg3VK8MhCJNAx"
4 }
```

- **Status Code:** 201 Created

CT02: Realizar Login

Objetivo: Usar o e-mail do passo anterior para logar e salvar o Token.

- **Método:** POST
- **URL:** `{{baseUrl}}/login`
- **Body (JSON):**

```
1 {
2   "email": "{{email_atual}}",
3   "password": "teste"
4 }
```

- **Resposta:**

```
1 {
2   "message": "Login realizado com sucesso",
3   "authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Imd1c3Rhdm9xdWVpcm96QHFhLmNvbSI6InBhc3N3b3JkIjoidGVzdGUiLCJpYXQiOi0jE3NzA2MTY3NzUsImV4cCI6MTc3MDYxOTUyMjE2fQ=="
4 }
```

- **Status Code:** 200 OK

CT03: Cadastrar Produto no Estoque

Objetivo: Criar o produto que será usado no fluxo de testes.

- **Método:** POST
- **URL:** `{{baseUrl}}/produtos`
- **Authorization:**

```
1 Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Imd1c3Rhdm9xdWVpcm96QHFhLmNvbSI6InBhc3N3b3JkIjoidGVzdGUiLCJpYXQiOi0jE3NzA2MTY3NzUsImV4cCI6MTc3MDYxOTUyMjE2fQ==
```

- **Body (JSON):**

```
1 {
2   "nome": "Produto Teste {{$randomInt}}",
3   "preco": 100,
4   "descricao": "Mouse Gamer criado via teste",
5   "quantidade": 10
6 }
```

- **Resposta:**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "oDbaxX7JG39MxrkG"
4 }//
```

- **Status Code:** 201 Created

CT04: Listar Produto via ID

Objetivo: Confirmar que o produto foi criado e está disponível na lista de busca por ID.

- **Método:** GET
- **URL:** `{{baseUrl}}/produtos/:id`

- **_id:** oDbaxX7JG39MrEKG

- **Resposta:**

```
1 {
2   "nome": "Produto Teste 448",
3   "preco": 100,
4   "descricao": "Mouse Gamer criado via teste",
5   "quantidade": 10,
6   "_id": "oDbaxX7JG39MrEK6"
7 }
```

- **Status Code:** 200 OK

CT05: Adicionar Item ao Carrinho

Objetivo: Simular a compra.

- **Método:** POST

- **URL:** `{{baseUrl}}/carrinhos`

- **Authorization:**

1 Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Imd1c3Rhdm9xdWVpcm96QHFhLmNvbSIsInBhc3N3b3JkIjoideGVzdGUlLCJpYXQiOjE3NzA2MTgwMDgsImV4cCI6MTc3MDYx

- **Body (JSON):**

```
1 {
2   "produtos": [
3     {
4       "idProduto": "0MmrBrQ9u8Eo5BGBv",
5       "quantidade": 1
6     }
7   ]
8 }
```

- **Resposta:**

```
1 {
2   "message": "Cadastro realizado com sucesso",
3   "_id": "AsEs9DvULn1QvyZb"
4 }
```

- **Status Code:** 201 Created

CT06: Conclure Compra (Checkout)

Objetivo: Finalizar o fluxo e validar a limpeza do carrinho.

- Método: DELETE

```
. URL: {{baseUrl}}/carrinhos/concluir-compra
```

- **Authorization:**

1 Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFpbCI6Imd1c3Rhdm9xdWVpcm96QHFhLmNvbSIsInBhc3N3b3JkIjoiaWoidGVzdGUiLCJpYXQiOjE3NzA2MTgwMDgsImV4cCI6MTc3MDYx

- **Body:** (Vazio)

• Resposta:

```
1 {
2   "message": "Registro excluído com sucesso. Estoque dos produtos reabastecido"
3 }
```